

Clean software development

Welcome!

Alessandro Sciarra
David A. Clarke

Lunch break

Clean software development

Day 1

Alessandro Sciarra

Z02 – Software Development Center

6 October 2025

Topics of the day

■ Formal welcome

1 Training format

2 The idea of clean code

3 Split and pick up

4 Clean testing (I)

5 Functional tests

6 Sum up and closing

A slow, but important start

Today you will start laying the fundament to become a better developer

Training format

Training schedule

	Monday 6	Tuesday 7	Wednesday 8	Thursday 9	Friday 10
9:00		Names, comments, ... Documentation Clean testing (II) Class, functions, IOSP	Git in real life	Data validation and data cleaning	Code review
10:00	Arrival to location	Break	Break	New project (I)	Review project from another group
11:00		Rename! Check comments Add tests for function Apply IOSP	Clean testing (IV)	New project (II)	Finalise review
12:00	Formal welcome		TDD live demo		Sum up reviews
13:00	Lunch	Lunch	Lunch	Lunch	Lunch
14:00	Welcome, week setup and Clean code	Clean testing (III)	Practice TDD (I)	Statistical analysis and plotting	Feedback and closing
15:00	Splitting & Pick-up Clean testing (I)	Improve your tests Add more tests		New project (III)	
16:00	Break	Break	Break	Break	
17:00	Testing framework and write Functional tests	DRY, KISS, YAGNI Refactoring!	Practice TDD (II) GitHub PR	New project (IV)	
18:00	Sum up and closing	Sum up and closing	Sum up and closing	Sum up and closing	Departure from location
19:00	Dinner	Dinner	Dinner	Dinner	
20:00		Informal discussion Office hour	Informal discussion Office hour	Informal discussion Office hour	

Our mental setup

- **Relaxed and respectful atmosphere**
↳ We are all here to learn!
- **It will be intense!**
- **More practice than theory** → Give your best!
- **It's going to be interactive** → ask, ask, ask!
- **This is a changing point, but yet only a starting one!**



Disclaimer

Slides are often full of text

↳ They are meant for offline reading!

[Clickable link to clickable ToC](#)



Alessandro – Who am I?

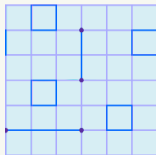
Ancient popular wisdom

«I am not saying to be Superman, I am just saying nobody has ever seen me and Superman together in the same room.»

- Member of the Software Development Center
→ Z02 project of the CRC-TR 211 collaboration
- A used-to-be “lattice practitioner”
- A dreamer who flatters himself that
 - physicists will one day properly use the IT tools they use
 - IT people will teach IT to physicists and work next to them
 - physicists will be able to focus on physics
- Not really an expert, but rather a self-thought software enthusiast



David – Who am I?



- Postdoc at Bielefeld
- Lattice practitioner studying QCD phase diagram, μ on $g = 2$
- Lots of experience developing high-performance lattice QCD code
 - [SIMULATEQCD](#)
 - [Grid](#)
- Data analysis: Collect generic tools in [AnalysisToolbox](#)
- Not really expert either
 - ↳ rather physicist who is paranoid about being wrong and perhaps guilty of improperly using IT tools sometimes 🙄

What to expect from this training



A scientist's point of view

The way you program today is better than it was yesterday and probably worse than it will be tomorrow! Have a good reason to do things the way you do them. It will happen that you will change mind and consequently your approach to the same problem. Again and over again, it is normal. Do not hesitate in doing so. To change mind and approach means to evolve and to become a better programmer.

Embrace a new working philosophy!



High-value material if you ever quit academia

A word about programming languages

How can we work and learn together if we use different languages?

- Most principles are language independent
- All used language in science are somewhat similar
- Trust your colleagues and focus on ideas, not technicalities

The Pragmatic Programmer

«Learn at least one new language every year. Different languages solve the same problems in different ways. By learning several different approaches, you can help broaden your thinking and avoid getting stuck in a rut.»

A. Hunt & D. Thomas

Be excited, never frustrated!

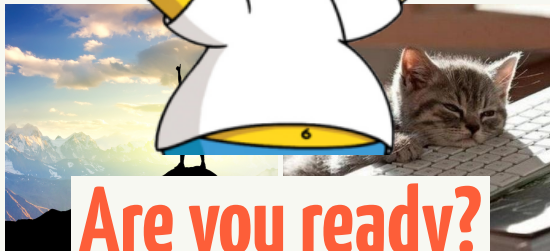
What do we expect from this training?

- We are aware that a lot will might be new
- Do not hesitate to ask!
- Speak up in case of need → It should never get frustrating!
- Put effort into the exercises
- The more active you are the more information will stay in your memory



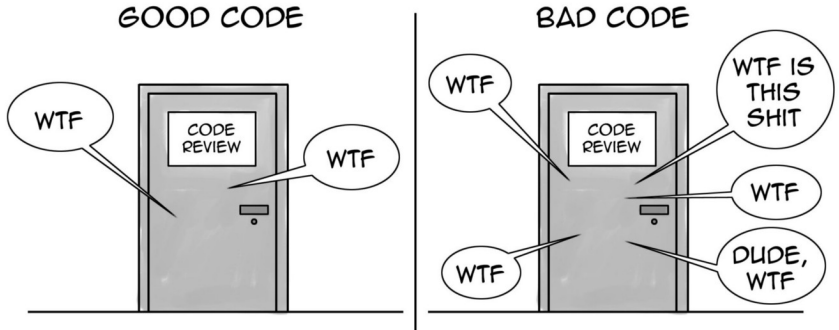
What do we expect from this training?

- We are aware that a lot will be new
- **Do not hesitate to ask!**
- Speak up in case of need **It should never be frustrating!**
- **Put effort into the exercise**
- The more active you are, the more it will stay in your memory



The idea of clean code

The best definition



THE ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE

Clean code

Clean code always looks like it was written by someone who cares. There is nothing obvious that you can do to make it better. [...] if you try to imagine improvements, you're led back to where you are.

Michael Feathers

I like my code to be elegant and efficient [...] clean code does one thing well.

Bjarne Stroustrup

Clean code is simple and direct. [...] Clean code reads like well-written prose.

Grady Booch

You know you are working on clean code when each routine you read turns out to be pretty much what you expected.

Ward Cunningham

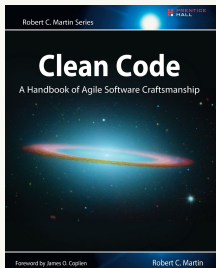


Clean code

Clean code always looks like it was written by someone who cares. There is nothing obvious that you can do to make it better. [...] if you try to imagine improvements, you're led back to where you are.

Michael Feathers

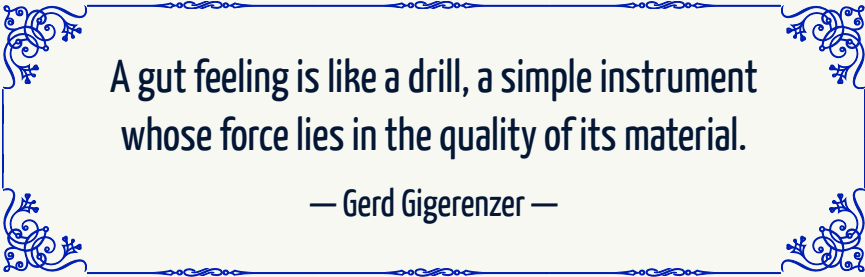
- Readable
- Maintainable
- Easy to extend
- Easy to use right
- Hard to break
- Testable/Tested
- ...



 The Clean Code developer road



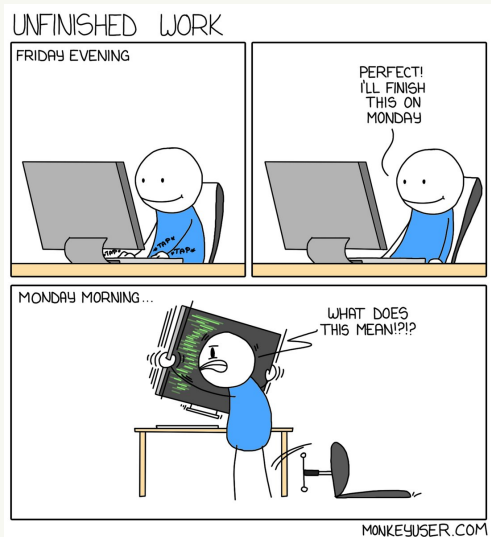
We will develop the needed code sense!



A gut feeling is like a drill, a simple instrument
whose force lies in the quality of its material.

— Gerd Gigerenzer —

Why do we need to care?



Why do we need to care?

- We never write code for one **time-invariant** reader!
- Learning and applying **few simple rules** increase quality of code base.
- When a program is modified, its complexity will increase, provided that one does not actively work against this.
- Many principles are totally **language independent**.



Why do we need to care?

The technical debt

It is a concept that reflects the implied cost of additional rework caused by choosing an easy solution now instead of using a better approach that would take longer.

Wikipedia

- We never write code for one **time-invariant** reader!
- Learning and applying **few simple rules** increase quality of code base.
- When a program is modified, its complexity will increase, provided that one does not actively work against this.
- Many principles are totally **language independent**.

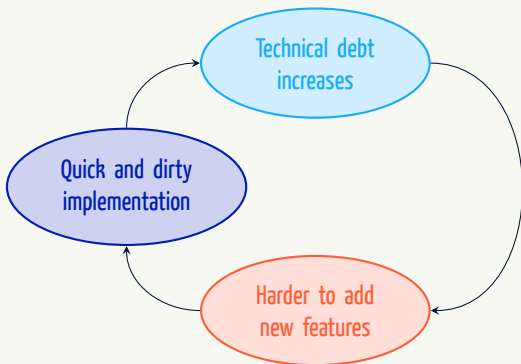


Why do we need to care?

The technical debt

It is a concept that reflects the implied cost of additional rework caused by choosing an easy solution now instead of using a better approach that would take longer.

Wikipedia

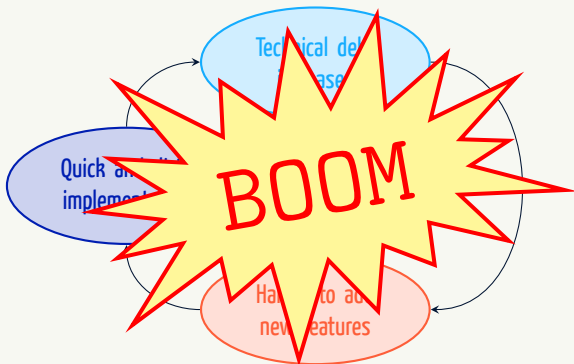


Why do we need to care?

The technical debt

It is a concept that reflects the implied cost of additional rework caused by choosing an easy solution now instead of using a better approach that would take longer.

Wikipedia



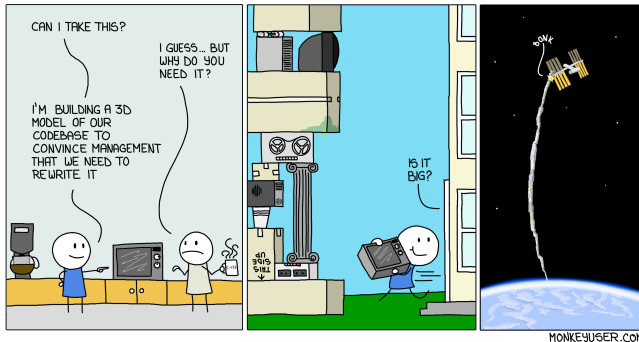
Why do we need to care?

The technical debt

It is a concept that reflects the implied cost of additional rework caused by choosing an easy solution now instead of using a better approach that would take longer.

[Wikipedia](#)

VISUALIZED CODEBASE

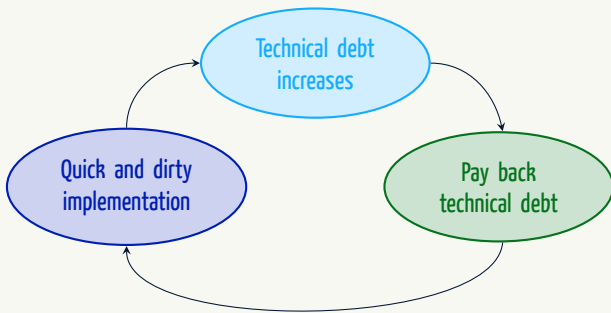


Why do we need to care?

The technical debt

It is a concept that reflects the implied cost of additional rework caused by choosing an easy solution now instead of using a better approach that would take longer.

Wikipedia



Technical debt naturally tends to increase



As scientist, why having high quality code?

- It is plenty of problems which need to be approached numerically
→ Programming is one of the major tasks in physics!
- Reproducibility should be a pillar of science
→ One of the main responsibilities of a scientist!
- Lots of bad code around and passed on across generations
→ A huge amount of time is wasted, subtracted to research!

It should not happen...

«Wow! There is definitely a new physics signal in these data!»

...few days/weeks/months/years later...

«Oh, no! It was just a bug in the code!»



The starting point...



The starting point...



Yours is without a doubt the worst code I've ever run



But it runs

Split and pick up

Sub-groups

- 1 Let's split in groups of 4 people.
 - Maximise number of programming languages per group



Sub-groups

- 1 Let's split in groups of 4 people.
 - Maximise number of programming languages per group
- 2 Take your snippet of code:
 - Each has 5 minutes to introduce it to the group.
 - Pick-up that which looks the least clean.



Sub-groups

- 1 Let's split in groups of 4 people.
 - Maximise number of programming languages per group
- 2 Take your snippet of code:
 - Each has 5 minutes to introduce it to the group.
 - Pick-up that which looks the least clean.
- 3 Why did you make such a choice?

How was it not clean?

Share the 3 most important reasons.

- Difficult to read (why?)
- Not maintainable
- Difficult to extend
- Difficult to use
- Easy to break
- Not tested (or testable)



Clean testing (I)

Why (automated) testing?

A mind-blowing changing point!

Lots of software should rather work...

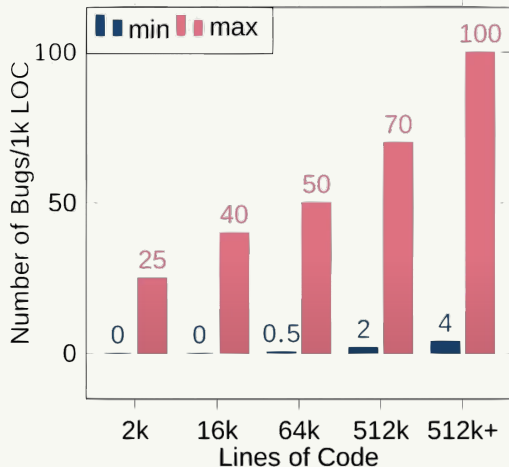
In our society you cannot spend 60 seconds without interacting with a software system

- 1 In modern car there is software that controls the steering wheel...
- 2 Think of medical devices, aeroplanes, lifts, etc.
- 3 Aren't you upset when an app on your phone stalls or crashes?!



An old, simple fact

Taken from Casper Jones, «Program quality and programmer productivity» – 1977



The unfortunate tendency in many cases

- It gives reasonable results $\Rightarrow$ it works! 🤖
- Manual (and possibly not systematic) testing is unfortunately still common practice in our community!
- Even if done in a systematic way, you will never run your tests enough!

A remarkable story about manual testing

I have been using an own manually tested “agenda-script” to track my working time **for months** and then one day it suddenly crashed. Where was the problem?



The unfortunate tendency in many cases

- It gives reasonable results $\Rightarrow$ it works! 🤖
- Manual (and possibly not systematic) testing is unfortunately still common practice in our community!
- Even if done in a systematic way, you will never run your tests enough!

A remarkable story about manual testing

I have been using an own manually tested “agenda-script” to track my working time **for months** and then one day it suddenly crashed. Where was the problem? I assumed **every** day has 24 hours... 😇



The unfortunate tendency in many cases

- It gives reasonable results $\Rightarrow$ it works! 🤖
- Manual (and possibly not systematic) testing is unfortunately still common practice in our community!
- Even if done in a systematic way, you will never run your tests enough!

Theorem:

Prove that, with $n, x, y, z \in \mathbb{N} > 0$ and $n > 2$, the equation $x^n + y^n = z^n$ has no solutions.

Would you accept this as proof or even argument?

$$2^3 + 3^3 \neq 4^3 \text{ and } 1^4 + 5^4 \neq 6^4 \text{ and } 4^5 + 2^5 \neq 5^5$$



The unfortunate tendency in many cases

- It gives reasonable results $\Rightarrow$ it works! 🤖
- Manual (and possibly not systematic) testing is unfortunately still common practice in our community!
- Even if done in a systematic way, you will never run your tests enough!

Now, software testing is not like mathematics...

...but would you accept a physics fact
without rigorous evidence?



Consider one of your programs. Does it work?

How do you prove that a software works? _____



Consider one of your programs. Does it work?

How do you prove that a software works?

You don't. You try proving it doesn't work... and fail!



Consider one of your programs. Does it work?

How do you prove that a software works?

You don't. You try proving it doesn't work... and fail!

The scientific method

- 1 Make an hypothesis $C \rightarrow$ «My code is correct»
- 2 Tests attempt to show $\neg C$
- 3 Confidence in C tracks thoroughness of tests

Testing code is science $\Leftrightarrow$ Popper's falsification principle



Uncle Bob's magic button



Uncle Bob's magic button

Fearless competence

Are you afraid to touch the code? Are you afraid to improve the code? Do you have that little echo in your ear saying – Ah, if it ain't broken, don't fix it? I expect you to improve the code, all of the time, fearless competence. How do you get fearless competence?



Tests! What if I had a button, a little button I could push on my keyboard and some lights would blink and science-fiction sounds would come out of my laptop for a few minutes and then a little green light would light up and that green light told me that the system worked. **And I believed it.**

Robert C. Martin



The wrong mental approach to testing

- I do not have time to waste writing tests.
- The result is reasonable, the code should be correct!
- This part of code is simple, why should I test it?
- Let me finish to implement this feature so that I can produce some data and in the meanwhile I write some tests.



<https://android.jelise.eu/basics-of-unit-testing-affdd2273310>

The wrong mental approach to testing

- I do not have time to test
- The results are not reliable
- This is not a real test
- Let's just produce some data and in the end we'll write some tests.

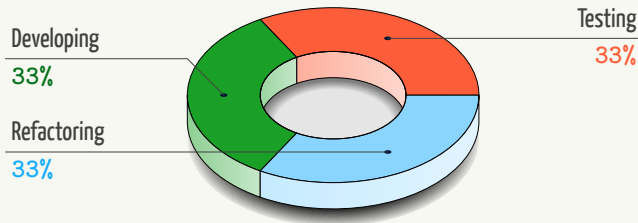
Please, NO!



<https://android.jlelse.eu/basics-of-unit-testing-affdd2273310>

The correct mental approach to testing

- Testing is part of the natural evolution of the code (not an overhead)
- Test code has to fulfil the same clean code rules as production code
- Tests are the key to be free to change (improve) the production code!
- Keep balance between adding new code, writing tests and refactoring



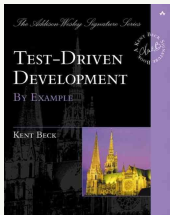
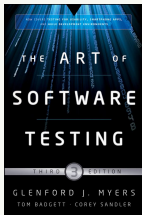
Different types of testing

Software testing

Test code is just as important as production code

- Testing levels

- Unit testing
- Integration testing
- System testing
- Acceptance testing



- Testing types and techniques

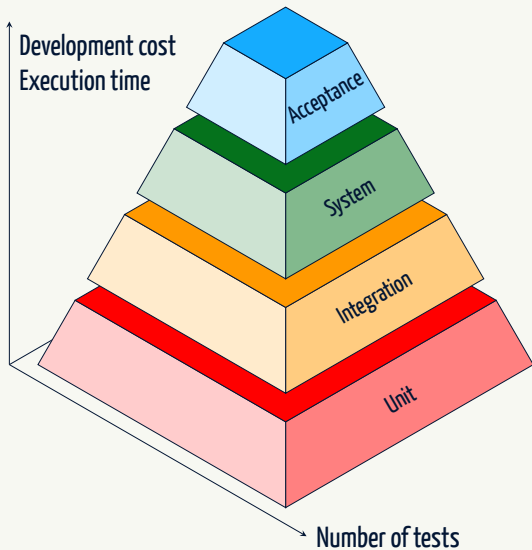
- Manual testing
- Automated testing
- Continuous testing
- Functional testing
- Mutation testing
- Fuzzy testing
- ...

- Plenty of videos/resources on the web

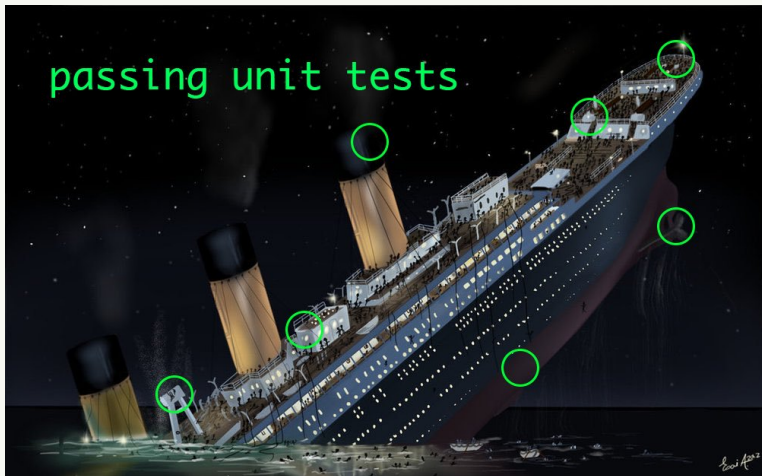
- 🔗 CppCon2015 – All your tests are terrible...
- 🔗 CppCon2020 – The science of unit tests
- 🔗 CppCon2020 – Back to Basics: Unit Tests
- 🔗 TDD, Where Did It All Go Wrong
- 🔗 TDD for those who don't need it



The tests pyramid



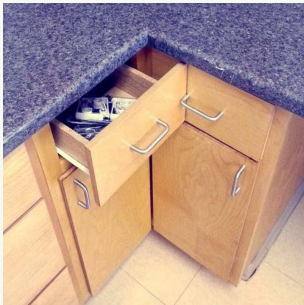
Unit tests are not enough!



Unit tests are not enough!

Integration tests

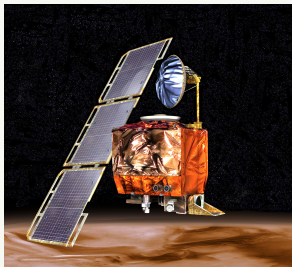
Once you are sure that all components work correctly alone, be sure they work as expected together!



Unit tests are not enough!

Integration tests

Once you are sure that all components work correctly alone, be sure they work as expected together!



 Mars Climate Orbiter lost in 1999 – By NASA/JPL/Corby Waste – © Public Domain



And once integration tests pass?

System testing

System testing tests a completely integrated system to verify that the system meets its requirements.

Wikipedia

Acceptance testing

Formal testing with respect to user needs, requirements, and business processes conducted to determine whether a system satisfies the acceptance criteria and to enable the user, customers or other authorized entity to determine whether to accept the system.

Standard Glossary of Terms used in Software Testing

In smaller projects

You might not need all categories, but, if you drop any, do it consciously!

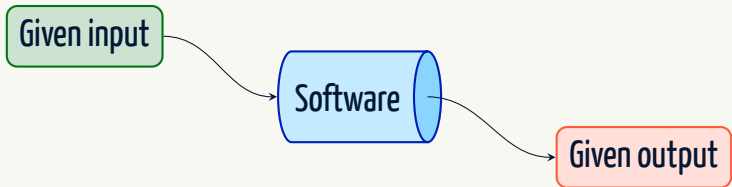


A technique for system/acceptance tests

Functional tests

They tend to answer the question of “can the user do this” or “does this particular feature work”.

Wikipedia



- Consider to implement these tests in an external script!
- This is a good starting point to work with legacy code without tests!



Testing frameworks

A small interlude: Unit test frameworks for C++

Essential for self-validation and automation!

1 Boost Test Library

 [Documentation](#)

```
#define BOOST_TEST_MODULE My Test
#include <boost/test/included/unit_test.hpp>

BOOST_AUTO_TEST_CASE(first_test)
{
    int i = 1;
    BOOST_TEST(i);
    BOOST_TEST(i == 2);
}
```

Running 1 test case...

```
test_file.cpp(8): error: in "first_test": check i == 2 has
    ↪ failed [1 != 2]
```

```
*** 1 failure is detected in the test module "My Test"
```



A small interlude: Unit test frameworks for C++

Essential for self-validation and automation!

1 Boost Test Library

 [Documentation](#)

2 GoogleTest

 [User's guide](#)

```
#include <gtest/gtest.h>
#include "src/factorial.h"

TEST(FactorialTest, HandlesZeroInput) {
    EXPECT_EQ(factorial(0), 1);
}

TEST(FactorialTest, HandlesPositiveInput) {
    EXPECT_EQ(factorial(1), 1);
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```



A small interlude: Unit test frameworks for C++

Essential for self-validation and automation!

1 Boost Test Library

 [Documentation](#)

2 GoogleTest or Catch2

 [User's guide](#) or  [GitHub project](#)

```
#include <catch2/catch_test_macros.hpp>

unsigned int factorial(unsigned int number)
{
    return number > 1 ? factorial(number - 1) * number : 1;
}

TEST_CASE("Factorials are computed", "[factorial]") {
    REQUIRE(factorial(0) == 1);
    REQUIRE(factorial(1) == 1);
    REQUIRE(factorial(3) == 6);
    REQUIRE(factorial(10) == 3628800);
}
```



A small interlude: Unit test frameworks for C++

Essential for self-validation and automation!

1 Boost Test Library

 [Documentation](#)

2 GoogleTest or Catch2

 [User's guide](#) or  [GitHub project](#)

3 Vir's Unit Test Framework

 [GitHub repository](#)

```
#include <vir/test.h>

TEST(test_name) {
    int test = 3;
    COMPARE(test, 2) << "more " << "details";
    //COMPARE(test, 2).on_failure("more ", "details");
}

FAIL:  at tests/testfile.cpp:5 (0x40451f):
FAIL:  test (3) == 2 (2) -> false more details
FAIL:  test_name

Testing done. 0 tests passed. 1 tests failed.
```

A small interlude: Unit test frameworks for C++

Essential for self-validation and automation!

1 Boost Test Library

 [Documentation](#)

2 GoogleTest or Catch2

 [User's guide](#) or  [GitHub project](#)

3 Vir's Unit Test Framework

 [GitHub repository](#)

...and so many mores!

 [List on Wikipedia](#)

Not all have the same weight and functionality

Pick your favourite depending on your needs and project:

1 Natural choice if you already depend on Boost

2 More complex but also more powerful

3 Very simple and lightweight



Break

Functional tests

A new begin

- 1 Choose a test framework for the language of your snippet
- 2 Reflect on the the difference between unit and functional tests
- 3 Set up the test framework and write **a dummy unit test**
- 4 Is the framework suitable for functional tests?
- 5 Add at least **one meaningful functional test** to your project
- 6 **Commit and push your code in a dedicated folder under**  **src**
 - ↳ Add a suitable `.gitignore` file → Check out [🔗this project](#)



Sum up and closing

Time to reflect on the day

- 1 Take few minutes to look back at the work done:
 - How many functional tests could you write?
 - Any challenging aspect?
 - Are there unanswered questions?
 - How was it overall?
 - Are you sceptical? If so, about what?
- 2 Report your group experience to the whole class.

